

Relatório Final do Projeto

Título: Tradução Automática de Tupi Antigo para Português Brasileiro com *Small Language Models* e *Fine-Tuning* Eficiente (LoRA/PEFT)

Autor: Miguel Conforto

Curso/Programa: Mestrado — PUC

Repositório: migconforto/slm_tupi

Data: julho de 2026

Resumo

Línguas de baixo recurso (*Low-Resource Languages*, LRLs) permanecem sub-representadas nos sistemas atuais de tradução automática, o que limita seu acesso a tecnologias de linguagem. Este projeto investiga a viabilidade de adaptar um *Small Language Model* (SLM) de 3 bilhões de parâmetros (**Qwen2.5-3B-Instruct**) para a tradução do **Tupi Antigo** para o **Português brasileiro**, um par extremamente escasso em dados paralelos. Adotou-se *fine-tuning* supervisionado (SFT) com adaptação de baixo posto (**LoRA/PEFT**) sobre um conjunto de dados aumentado de 2.012 pares de sentenças. O treinamento foi conduzido em hardware de baixo consumo (NVIDIA RTX 2060, 6 GB), demonstrando que a *perda* de treino cai de forma consistente (de $\approx 2,71$ para $\approx 0,66$ ao longo de três épocas, com perda média final de 1,27). Os resultados estão em produção. O trabalho contribui com um *pipeline* reprodutível de SFT e avaliação e discute os próximos passos para uma avaliação quantitativa formal.

Palavras-chave: tradução automática; línguas de baixo recurso; Tupi; LoRA; PEFT; *Small Language Models*; destilação de conhecimento.

1. Introdução

1.1 Contexto e motivação

Os avanços recentes em *Large Language Models* (LLMs) e em Tradução Automática Neural (NMT) melhoraram substancialmente a qualidade de tradução para idiomas de alto recurso. Contudo, persiste uma disparidade acentuada para línguas de baixo recurso, tanto por falta de dados paralelos quanto por sub-representação nos corpora de pré-treinamento. O **Tupi Antigo**, língua histórica de grande importância cultural e linguística no Brasil, é um caso extremo: praticamente não há sistemas de tradução automática dedicados e os recursos digitais são fragmentados.

1.2 Problema

Modelos grandes o suficiente para traduzir bem costumam ser inviáveis em cenários com restrição de recursos (privacidade, custo, hardware limitado). A questão central deste projeto é: **é possível ajustar um modelo pequeno (3B), em hardware de consumo, para produzir traduções úteis de Tupi Antigo para Português?**

1.3 Objetivos

Objetivo geral: avaliar a adaptação de um SLM para a tradução Tupi-Português por meio de *fine-tuning* eficiente.

Objetivos específicos:

1. Construir um *pipeline* reproduzível de pré-processamento, *fine-tuning* (SFT com LoRA) e inferência para o par Tupi-Português.
 2. Treinar o Qwen2.5-3B-Instruct sobre um conjunto aumentado de pares Tupi/Português.
 3. Analisar o comportamento do treinamento (curva de perda, custo computacional) em hardware limitado.
 4. Avaliar qualitativamente as traduções geradas e estabelecer a base para uma avaliação quantitativa futura.
-

2. Fundamentação teórica e trabalhos relacionados

O projeto se baseia no artigo "*Are Small Language Models the Silver Bullet to Low-Resource Languages Machine Translation?*" (LoResMT 2026), que mostra que a **destilação de conhecimento** a partir de modelos-professores fortes, usando predominantemente dados monolíngues da língua-alvo, pode elevar a qualidade de tradução de SLMs a ponto de igualar ou superar sistemas muito maiores.

Conceitos-chave utilizados:

- **Supervised Fine-Tuning (SFT):** ajuste do modelo a pares (entrada, saída) formatados como diálogo instruído, usando o *template* de *chat* do próprio modelo.
- **LoRA (Low-Rank Adaptation):** técnica de *fine-tuning* paramétrico-eficiente que congela os pesos originais e treina apenas matrizes de baixo posto injetadas nas camadas de atenção e MLP, reduzindo drasticamente memória e custo.
- **PEFT (Parameter-Efficient Fine-Tuning):** família de métodos, aqui representada pela biblioteca `peft` da Hugging Face combinada ao `SFTTrainer` da biblioteca `trl`.

3. Metodologia

3.1 Dados

O conjunto de treino é o `dataset_augmentated.json`, com **2.012 pares** de sentenças, em que a coluna `input` contém o texto em Tupi e `output`, a tradução em Português.

O pré-processamento (`utils_train.pre_process`) calcula o comprimento das entradas e mantém a estrutura tabular; filtros opcionais para "alucinações" estão previstos, mas desativados nesta configuração. Após embaralhamento (semente 42), o conjunto foi dividido em **70% treino (1.408 exemplos)** e **30% validação (604 exemplos)**.

Cada exemplo é convertido em um *prompt* instruído com o *template* de *chat* do modelo:

```
<|im_start|>system
Você é um assistente de IA muito útil para traduções.<|im_end|>
<|im_start|>user
Traduza o seguinte texto em Tupi para Português. Não inclua
informações
adicionais ou conteúdo irrelevante.

k unhâmukuetá îkó ka'ape<|im_end|>
<|im_start|>assistant
moças vivem na mata<|im_end|>
```

3.2 Modelo e configuração de treinamento

Item	Valor
Modelo base	Qwen/Qwen2.5-3B-Instruct (decoder-only, 36 camadas, hidden 2048)
Método	LoRA (PEFT)
Posto LoRA (r)	256 (experimento principal); variantes com $r \in \{8, 16, 128\}$
<code>lora_alpha</code> / <code>dropout</code>	8 / 0

Item	Valor
Módulos-alvo	<code>q, k, v, o_proj, gate, up, down_proj</code>
Parâmetros treináveis	$\approx 239,5$ M (7,20% de 3,33 B)
Épocas	3
<i>Learning rate</i>	$1e-5$
<i>Scheduler</i>	coseno, <code>warmup_ratio = 0,5</code>
<i>Batch</i> (treino/aval.)	1 / 1
<code>max_seq_length</code>	128
<code>weight_decay / max_grad_norm</code>	0,01 / 0,3
Precisão	fp32 (fp16/bf16 desabilitados)
Semente	42
<i>Framework</i>	Hugging Face <code>transformers + peft + trl</code> (SFTTrainer), <i>logging</i> TensorBoard

3.3 Ambiente

- **GPU:** NVIDIA GeForce RTX 2060, 6 GB VRAM.
- **Software:** PyTorch 2.5.1 + CUDA 12.1; Python 3.10; Windows.

Por ser um modelo de 3B em fp32, o consumo de memória excedeu a VRAM física (pico reportado de $\approx 17,7$ GB), o que implicou uso de memória compartilhada/*offload* — principal fator do tempo de treino elevado.

3.4 Avaliação

A inferência (`EVAL_SFT.py`) carrega o modelo base com o adaptador LoRA (`checkpoint-4224`) e gera traduções do conjunto de validação, com quantização em 4-bit (NF4) para caber em memória. Parâmetros de geração: `temperature = 0,1`, `top_p = 0,9`, `max_new_tokens = 50`, amostragem ativada. As saídas são exportadas para `predictions.csv`.

4. Desenvolvimento e implementação

O *pipeline* foi organizado em módulos reutilizáveis:

- **Preparação de dados e *prompts*** (`utils/utils_train.py`): `pre_process`, `create_prompt` (template Qwen/ChatML) e `create_prompt_gemma` (template Alpaca), além de utilitários de contagem de parâmetros treináveis.
 - **Treinamento** (`script/SFT.py`): montagem do *dataset* tokenizado, configuração LoRA, `SFTConfig/SFTTrainer` e laço de treino com avaliação periódica e salvamento por época.
 - **Inferência** (`script/EVAL_SFT.py`): quatro estratégias — *pipeline* local da Hugging Face.
-

5. Resultados e discussão

5.1 Dinâmica do treinamento

A perda de treino apresentou queda consistente ao longo dos 4.224 passos (3 épocas):

Passo	Época	*Loss* de treino	*Learning rate*
1000	0,71	2,706	4,73e-6
2000	1,42	1,069	9,47e-6
3000	2,13	0,796	6,24e-6
4000	2,84	0,657	2,75e-7
—	3,00	1,274 (média)	—

A trajetória decrescente indica que o modelo aprendeu efetivamente o mapeamento Tupi-Português a partir do conjunto aumentado, sem sinais de divergência. O *warmup* longo (50%) combinado ao *scheduler* cosseno explica o comportamento do *learning rate*.

5.2 Custo computacional

- **Tempo total de treino:** ≈ 66.255 s (≈ 18 h 24 min).
- **Velocidade:** $\approx 0,064$ passos/s (limitada pelo *offload* de memória).
- **Pico de memória reservada:** $\approx 17,7$ GB (contra 6 GB de VRAM física).

O gargalo principal foi a memória: rodar um modelo de 3B em fp32 numa GPU de 6 GB obriga o uso de memória do sistema, penalizando a velocidade.

5.3 Análise qualitativa

Exemplos do conjunto de validação (Tupi-referência em Português):

Tupi	Referência
Agoacem apyába cetà;	Achei muitos índios.
abá guyra'i oîpsyky ka'ape	O homem capturou o passarinho na mata
Oiecuáb Apyàba cetá,	Apareceram muitos índios.
Çupí na xe anga recé rüã.	e não por minha alma.

As traduções geradas foram exportadas para `predictions.csv` para inspeção manual. A avaliação das métricas (BLEU, chrF e afins) e a análise qualitativa serão publicadas em breve.

5.4 Limitações

- **Volume de dados reduzido:** 2.012 pares.
- **Restrição de hardware**, que limitou *batch size*, comprimento de sequência e velocidade.

6. Conclusão e trabalhos futuros

Sob elaboração.

Trabalhos futuros:

1. Comparar sistematicamente os postos LoRA ($r \in \{8, 16, 128, 256\}$) e o *full fine-tuning*.
 2. Ampliar e curar o corpus Tupi-Português, incluindo verificação por falantes/estudiosos.
 3. Avaliar a tradução bidirecional (português-tupi) e o uso do módulo de RAG com dicionário/gramática.
-

Referências

1. Song, Y.; Li, L.; Lothritz, C.; Ezzini, S.; Sleem, L.; Gentile, N.; State, R.; Bissyandé, T. F.; Klein, J. *Are Small Language Models the Silver Bullet to Low-Resource Languages Machine Translation?* Proceedings for the Ninth Workshop on Technologies for Machine Translation of Low Resource Languages (LoResMT 2026), Rabat, Morocco: ACL, 2026. Disponível em: <https://aclanthology.org/2026.loresmt-1.1/>
 2. Hu, E. J. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
 3. Qwen Team. *Qwen2.5 Technical Report*. 2024.
 4. NLLB Team et al. *No Language Left Behind: Scaling Human-Centered Machine Translation*. 2022 (benchmark FLORES-200).
 5. Hugging Face. Documentação das bibliotecas `transformers`, `peft`, `trl` e `datasets`.
-